Transformações 3D – Parte 3
Programação em Three.js

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

10 de maio de 2026

Sumário

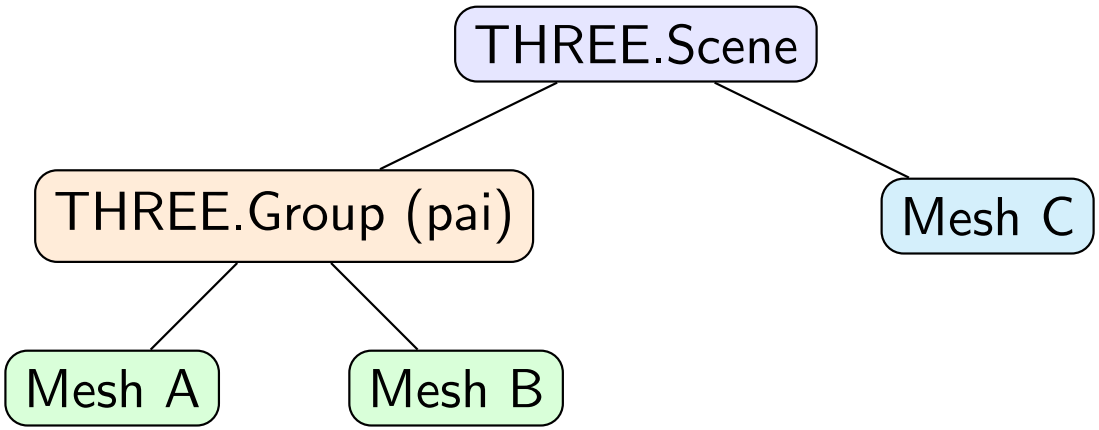
1. OpenGL vs Three.js
   - Equivalência de conceitos
2. Exemplo 1 – Rotacao interativa
   - Esfera e chapa com teclado
3. Exemplo 2 – Controle de escopo
   - Grupos independentes
4. Exemplo 3 – Sistema solar
   - Animacao com hierarquia
5. Exercicio
   - Cubo e torus com selecao

De OpenGL para Three.js

OpenGL / GLUT	Three.js (JavaScript)
`glTranslatef(tx,ty,tz)`	`mesh.position.set(tx,ty,tz)`
`glRotatef(ang,x,y,z)`	`mesh.rotation.y = degToRad(ang)`
`glScalef(sx,sy,sz)`	`mesh.scale.set(sx,sy,sz)`
`glPushMatrix()`	`const g = new THREE.Group()`
`glPopMatrix()`	(fim do escopo do grupo)
`scene.add(group)`	`scene.add(g)`
`glutSolidSphere`	`THREE.SphereGeometry`
`glutSolidCube`	`THREE.BoxGeometry`
`glutWireSphere`	`wireframe: true` no material
`glutTimerFunc`	`requestAnimationFrame`
`glutSpecialFunc`	`addEventListener('keydown', ...)`
`glFlush / glSwapBuffers`	`renderer.render(scene, camera)`

Hierarquia Three.js

Em Three.js, toda cena é uma árvore de objetos. A hierarquia controla o **escopo das transformações** (equivalente ao push/pop matrix do OpenGL):



- Transformações num `Group` afetam **todos os filhos**
- Transformações num `Mesh` afetam **só ele**
- **Grupos aninhados** permitem composições complexas (sistema solar, braços robóticos etc.)

Setup básico de uma cena Three.js

```
1  // 1. Cena, camera e renderer
2  const scene = new THREE.Scene();
3  const camera = new THREE.PerspectiveCamera(
4                   45, // campo de visao
5                   window.innerWidth / window.innerHeight,
6                   0.1, 1000); // near, far
7  camera.position.set(0, 2, 10);
8
9  const renderer = new THREE.WebGLRenderer({antialias: true});
10 renderer.setSize(window.innerWidth, window.innerHeight);
11 document.body.appendChild(renderer.domElement);
12
13 // 2. Loop de renderizacao
14 function animate() {
15   requestAnimationFrame(animate);
16   renderer.render(scene, camera);
17 }
18 animate();
```

Observacao

`PerspectiveCamera` gera perspectiva realista. Para projecao ortogonal, use `THREE.OrthographicCamera`.

Exemplo 1: o que vamos construir

Equivalente ao Exemplo 1 do OpenGL:

- Uma **esfera wireframe** amarela centralizada na origem
- Uma **chapa azul** (cubo achatado) posicionada abaixo
- **Setas do teclado**: rotacionam os dois objetos juntos em x e y
- **Page Up / Page Down**: zoom in e zoom out (escala uniforme)

Conceito-chave

Um `THREE.Group` agrupa os dois objetos. Transformar o grupo afeta ambos simultaneamente — equivalente a fazer as transformações **fora** do push/pop em OpenGL.

Exemplo 1 – Criando os objetos

```
1  // Grupo que contera os dois objetos
2  const grupo = new THREE.Group();
3  scene.add(grupo);
4
5  // Esfera wireframe amarela (raio 1, 20x20 divisoes)
6  const esfera = new THREE.Mesh(
7    new THREE.SphereGeometry(1, 20, 20),
8    new THREE.MeshBasicMaterial({color:0xffff00, wireframe:true})
9  );
10 grupo.add(esfera);
11
12 // Chapa azul: cubo escalado (4x0.2x4) posicionado abaixo
13 const chapa = new THREE.Mesh(
14   new THREE.BoxGeometry(4, 0.2, 4),
15   new THREE.MeshBasicMaterial({color:0x0000ff})
16 );
17 chapa.position.y = -1.1;
18 grupo.add(chapa);
```

Exemplo 1 – Controles de teclado

```
let alfa = 0, beta = 0, delta = 1;

document.addEventListener('keydown', (e) => {
  switch(e.key) {
    case 'ArrowRight': beta -= 1; break;
    case 'ArrowLeft': beta += 1; break;
    case 'ArrowUp': alfa -= 1; break;
    case 'ArrowDown': alfa += 1; break;
    case 'PageUp': delta *= 1.1; break;
    case 'PageDown': delta *= 0.9; break;
  }
});

// No loop de animacao:
function animate() {
  requestAnimationFrame(animate);
  const toRad = THREE.MathUtils.degToRad;
  grupo.rotation.x = toRad(alfa);
  grupo.rotation.y = toRad(beta);
  grupo.scale.setScalar(delta); // escala uniforme
  renderer.render(scene, camera);
}
```

Exemplo 2: controle de escopo com Groups

Equivalente ao Exemplo 2 do OpenGL (push/pop matrix):

- Duas esferas wireframe: **vermelha** (direita) e **amarela** (esquerda)
- As **rotações** afetam cada esfera **de forma independente**
- A **escala** (Page Up/Down) afeta as **duas** simultaneamente

Analogia com OpenGL

OpenGL	**Three.js**
`glPushMatrix()`	`new THREE.Group()`
transformações locais	`group.rotation`, `.position`
`glPopMatrix()`	(sair do escopo do grupo)
fora do push/pop	transformar o grupo pai

Exemplo 2 – Hierarquia de grupos

```
// Grupo global: escala afeta as duas esferas
const globalGroup = new THREE.Group();
scene.add(globalGroup);

const geo = new THREE.SphereGeometry(1, 20, 20);

// Esfera vermelha (direita) -- grupo proprio
const grupoVerm = new THREE.Group();
const esfVerm = new THREE.Mesh(geo,
  new THREE.MeshBasicMaterial({color:0xff0000, wireframe:true}));
esfVerm.position.x = 3;
grupoVerm.add(esfVerm);
globalGroup.add(grupoVerm);

// Esfera amarela (esquerda) -- grupo proprio
const grupoAmar = new THREE.Group();
const esfAmar = new THREE.Mesh(geo,
  new THREE.MeshBasicMaterial({color:0xffff00, wireframe:true}));
esfAmar.position.x = -3;
grupoAmar.add(esfAmar);
globalGroup.add(grupoAmar);
```

Exemplo 2 – Animacao e controles

```
let alfa = 0, beta = 0, delta = 1;

document.addEventListener('keydown', (e) => {
  // mesmo tratamento de teclas do Exemplo 1
  if(e.key === 'ArrowRight') beta -= 1;
  if(e.key === 'ArrowLeft') beta += 1;
  if(e.key === 'ArrowUp') alfa -= 1;
  if(e.key === 'ArrowDown') alfa += 1;
  if(e.key === 'PageUp') delta *= 1.1;
  if(e.key === 'PageDown') delta *= 0.9;
});

function animate() {
  requestAnimationFrame(animate);
  const toRad = THREE.MathUtils.degToRad;

  // Rotacoes afetam cada esfera pelo seu grupo proprio
  grupoVerm.rotation.set(toRad(alfa), toRad(beta), 0);
  grupoAmar.rotation.set(toRad(alfa), toRad(beta), 0);

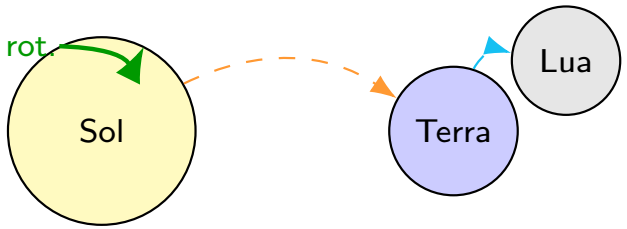
  // Escala via grupo global: afeta as duas
  globalGroup.scale.setScalar(delta);

  renderer.render(scene, camera);
}
animate();
```

Exemplo 3: sistema solar simplificado

Simplificacoes astronômicas usadas:

- Sol: gira em torno do proprio eixo a cada **28 dias**
- Terra: orbita o Sol em **360 dias** e gira em torno do proprio eixo em **24 horas**
- Lua: orbita a Terra a cada **28 dias**
- Tamanhos e distâncias não são proporcionais à realidade



Objetos em wireframe pois a iluminacao ainda nao foi estudada.

Hierarquia de grupos

`scene` → `grupoSol` → `grupoTerra` → `lua`

Exemplo 3 – Sol e estrutura base

```
1  const grupoSol = new THREE.Group();
2  scene.add(grupoSol);
3
4  // Sol: esfera wireframe amarela, posicionada na origem
5  const sol = new THREE.Mesh(
6    new THREE.SphereGeometry(1, 32, 32),
7    new THREE.MeshBasicMaterial({color:0xffff00, wireframe:true})
8  );
9  grupoSol.add(sol);
10
11 // grupoTerra e filho do grupoSol:
12 // quando grupoSol girar, ele carrega a Terra junto (orbita)
13 const grupoTerra = new THREE.Group();
14 grupoSol.add(grupoTerra);
```

Exemplo 3 – Terra e Lua

```
1  // Terra: deslocada 4 unidades do Sol
2  const terra = new THREE.Mesh(
3    new THREE.SphereGeometry(0.5, 20, 20),
4    new THREE.MeshBasicMaterial({color:0x4488ff, wireframe:true})
5  );
6  terra.position.x = 4;
7  grupoTerra.add(terra);
8
9  // Lua: filha de grupoTerra, orbita junto com a Terra
10 const lua = new THREE.Mesh(
11   new THREE.SphereGeometry(0.2, 12, 12),
12   new THREE.MeshBasicMaterial({color:0xaaaaaa, wireframe:true})
13 );
14 lua.position.x = 5; // 1 unidade alem da Terra
15 grupoTerra.add(lua);
```

Exemplo 3 – Loop de animacao com tempo

```
let hora = 0, dia = 0;
const toRad = THREE.MathUtils.degToRad;

function animate() {
  requestAnimationFrame(animate);

  // Avanca o relogio: a cada frame = 1 hora
  hora += 1;
  if(hora >= 24) { hora = 0; dia += 1; }
  if(dia >= 360) { dia = 0; }

  // Sol: rotacao propria (28 dias por volta)
  grupoSol.rotation.y = toRad(dia * 360 / 28);

  // Terra: orbita em torno do Sol (1 volta/360 dias)
  grupoTerra.rotation.y = toRad(dia);

  // Terra: rotacao propria em torno do proprio eixo
  terra.rotation.y = toRad(hora * 360 / 24);

  // Lua: orbita em torno da Terra (28 dias)
  // Nota: lua.position.x ja a desloca da Terra;
  // a rotacao do grupoTerra ja a carrega na orbita.
  // Orbita adicional propria da Lua em torno da Terra:
  lua.position.x = 4 + Math.cos(toRad(dia * 360/28)) * 1;
  lua.position.z = Math.sin(toRad(dia * 360/28)) * 1;

  renderer.render(scene, camera);
}
animate();
```

Exemplo 3 – Camera e configuracao da cena

```
1  // Camera posicionada acima para ver o plano orbital
2  const camera = new THREE.PerspectiveCamera(45, 1, 0.1, 1000);
3  camera.position.set(0, 8, 0); // acima
4  camera.lookAt(0, 0, 0); // olha para a origem (Sol)
5  camera.up.set(1, 0, 0); // "cima" da camera e o eixo x
6
7  // Fundo preto
8  renderer.setClearColor(0x000000);
9
10 // Luz ambiente (necessaria para MeshStandardMaterial;
11 // MeshBasicMaterial nao precisa de luz)
12 // Para wireframe: MeshBasicMaterial suficiente.
```

Equivalência: double buffering

Em Three.js o double buffering é **automatico**: o `WebGLRenderer` usa dois framebuffers por padrão. Não é necessária nenhuma configuração extra (equivalente ao `GLUT_DOUBLE` + `glutSwapBuffers`).

Exercicio: selecao de objeto por tecla

Enunciado

Crie uma pagina HTML com Three.js contendo **dois objetos lado a lado**: um **cubo wireframe** e um **torus wireframe** (equivalente ao bule do OpenGL).

- Pressionar **tecla 1**: seleciona o cubo (as setas rotacionam o cubo)
- Pressionar **tecla 2**: seleciona o torus (as setas rotacionam o torus)
- Seta direita/esquerda: rotacao horaria/antihoraria em torno do **centroide do objeto selecionado**
- O cubo deve sempre rotacionar em torno do eixo $(1, 1, 1)$ (normalizado)
- O torus deve sempre rotacionar em torno do eixo $(1, 0, 0)$

Dica: eixo de rotacao arbitrario

Use `object.rotateOnAxis(axisVec3, angle)` onde `axisVec3` e um `THREE.Vector3` **normalizado**.

Exercicio – Dica: estrutura base

```
let selecionado = 1; // 1 = cubo, 2 = torus
const eixoCubo = new THREE.Vector3(1,1,1).normalize();
const eixoTorus = new THREE.Vector3(1,0,0);

document.addEventListener('keydown', (e) => {
  if(e.key === '1') selecionado = 1;
  if(e.key === '2') selecionado = 2;

  const vel = 0.05; // velocidade em radianos
  if(e.key === 'ArrowRight') {
    if(selecionado === 1)
      cubo.rotateOnAxis(eixoCubo, vel);
    else
      torus.rotateOnAxis(eixoTorus, vel);
  }
  if(e.key === 'ArrowLeft') {
    if(selecionado === 1)
      cubo.rotateOnAxis(eixoCubo, -vel);
    else
      torus.rotateOnAxis(eixoTorus, -vel);
  }
});
```

Exercicio – Dica: criando os objetos

```
1  // Cubo wireframe
2  const cubo = new THREE.Mesh(
3    new THREE.BoxGeometry(1.5, 1.5, 1.5),
4    new THREE.MeshBasicMaterial({color:0x00aaff, wireframe:true})
5  );
6  cubo.position.x = -2;
7  scene.add(cubo);
8
9  // Torus wireframe (equivalente ao bule do OpenGL)
10 const torus = new THREE.Mesh(
11   new THREE.TorusGeometry(0.8, 0.3, 16, 50),
12   new THREE.MeshBasicMaterial({color:0xff6600, wireframe:true})
13 );
14 torus.position.x = 2;
15 scene.add(torus);
16
17 // Loop de renderizacao (sem animacao automatica,
18 // os objetos so giram ao pressionar teclas)
19 function animate() {
20   requestAnimationFrame(animate);
21   renderer.render(scene, camera);
22 }
23 animate();
```